$$
\begin{array}{ll}
\mathrm{Num}(sz) & ::= \{\, i \mid 0 \le i < 2^{sz} \,\} \\
\mathrm{Time} & ::= \mathbb{N} \\
\mathrm{BlockID} & ::= \mathbb{N} \\
\mathrm{CallID} & ::= \mathbb{N} \\
\mathrm{Mem} & ::= \mathrm{Time} \times (\mathrm{BlockID} \nrightarrow \mathrm{Block}) \times (\mathrm{CallID} \nrightarrow \mathrm{Time} \uplus \{\mathbf{None}\}) \\
\mathrm{AddrSpace} & ::= \mathbb{N} \\
\mathrm{Block} & ::= \{\, (t, r, n, a, c, P) \mid t \in \{\, \mathtt{stack}, \mathtt{heap}, \mathtt{global}, \mathtt{function} \,\} \wedge r \in (\mathrm{Time} \times (\mathrm{Time} \uplus \{\infty\})) \wedge \\
& \qquad n \in \mathbb{N} \wedge a \in \mathbb{N} \wedge c \in \mathrm{Byte}^n \wedge P \in (\mathrm{AddrSpace} \nrightarrow \mathrm{Num}(64)^{N+1}) \,\} \\
\mathrm{LogAddr}(s) & ::= \{\, \mathrm{Log}(l, o, s) \mid l \in \mathrm{BlockID} \wedge o \in \mathrm{Num}(\mathrm{ptrsz}(s)) \,\} \\
\mathrm{PhyAddr}(s) & ::= \{\, \mathrm{Phy}(o, s, I, cid) \mid o \in \mathrm{Num}(\mathrm{ptrsz}(s)) \wedge I \subset \mathrm{Num}(\mathrm{ptrsz}(s)) \wedge cid \in \mathrm{CallID} \uplus \{\mathbf{None}\} \,\} \\
\mathrm{Addr}(s) & ::= \mathrm{LogAddr}(s) \uplus \mathrm{PhyAddr}(s) \\
[\![\mathbf{i}sz]\!] & ::= \mathrm{Num}(sz) \uplus \{\, \mathbf{poison} \,\} \\
[\![\langle sz \times ty \rangle]\!] & ::= \{0, \ldots, sz-1\} \to [\![ty]\!] \\
[\![ty*]\!] & ::= \mathrm{Addr}(0) \uplus \{\, \mathbf{poison} \,\} \\
\mathrm{Name} & ::= \{\, \mathtt{\%x}, \mathtt{\%y}, \ldots \,\} \\
\mathrm{Reg} & ::= \mathrm{Name} \to \{\, (ty, v) \mid v \in [\![ty]\!] \,\} \\
\mathrm{Byte} & ::= \mathrm{Bit}^8 \\
\mathrm{Bit} & ::= [\![\mathbf{i}1]\!] \uplus \mathrm{AddrBit} \\
\mathrm{AddrBit} & ::= \{\, (p, i) \mid \exists s.\ p \in \mathrm{Addr}(s) \wedge (0 \le i < \mathrm{ptrsz}(s)) \,\}
\end{array}
$$

Fig. 4. Definitions. $\mathrm{ptrsz}(s)$ is the pointer size (in bits) for a given address space s (e.g., 64). The set of all possible values of a type ty is given by $[\![ty]\!]$.

dependence tracking, since that would block standard integer optimizations such as equality propagation. We employ two new techniques to recover precision instead: delayed bounds checking (to restrict the set of objects a pointer can point to, while keeping pointer arithmetic operations pure), and twin memory allocation (to prevent address guessing).

4 SEMANTICS AND TRANSFORMATIONS

In this section, we present a formal view of the modified memory model for LLVM that we informally presented in Section 3. Our top-level design goal was to support low-level operations required by C and C++, such as casting between integers and pointers, while also enabling high-level memory optimizations. Additional goals were: not interfering with integer optimizations, not constraining opportunities for code motion, not requiring major changes to make LLVM conform to the new model, and finally, avoiding significant regressions in compile time and in quality of generated code.

4.1 Logical and Physical Pointers

As we saw in Section 2.4, we have two types of pointers. *Logical* pointers are obtained by calling an allocation function or by doing pointer arithmetic on a logical pointer. In Section 2.4 these are the pointers with provenance of one object, e.g., `(val=0x10, obj=p)`. The second type of pointer is the *physical* pointer, which is the result of an integer-to-pointer cast. In Section 2.4, these are the pointers with wildcard provenance, e.g., `(val=0x10, obj=*)`; they can access any object.

In Fig. 4 we show the definitions for our model. Logical and physical pointers are represented, respectively, by $\mathrm{Log}(l, o, s)$ and $\mathrm{Phy}(o, s, I, cid)$.

Logical Pointers. A logical pointer $\mathrm{Log}(l, o, s)$ consists of a logical block id l, an offset o within the block, and the address space s it corresponds to (explained later in Section 4.2). A logical pointer corresponds to the address $P + o$ on the physical machine, where P is the base address of block l.